

Chapter 12

角色動畫 (Animator)

Horazon

手機遊戲開發

複習：上週重點

- [x] 實現了 Jump Force 跳躍。
- [x] 解決了無限跳躍問題 (Ground Check)。
- [x] 調整了重力與手感。

現在主角能跑能跳，但看起來還是「滑來滑去」的僵屍。
今天我們要賦予他**生命力**！

本章目標

1. 理解 Unity 動畫系統 (Mecanim)。
2. 製作關鍵動作：Idle (待機), Run (跑), Jump (跳)。
3. 設定 Animator Controller (狀態機)。
4. 透過程式 (`animator.SetFloat`) 控制動畫切換。

必備素材

請檢查你的素材包，是否包含主角的分解動作圖？

通常會有：

- `Player_Idle.png` (或是多張 `Idle_01`, `Idle_02`...)
- `Player_Run.png` (多張連環圖)
- `Player_Jump.png`

(如果你是用單張大圖 *Sprite Sheet*，請先用 *Sprite Editor* 切割好)

製作動畫片段 (Animation Asset)

1. 選取場景中的 **Player**。
2. 開啟 **Animation 視窗** (Window -> Animation -> Animation)。
3. 點選視窗中間的 **Create** 按鈕。
4. 建立第一支動畫：命名為 `Player_Idle`，存在 `Assets/Animations/` 資料夾。

錄製動畫：Idle (待機)

1. 在 Animation 視窗左側，你可以看到 **Player_Idle** 。
2. 選取 Project 裡的所有 Idle 圖片 (例如 Idle_0 到 Idle_5) 。
3. 直接**拖曳**到 Animation 視窗的時間軸上。
4. 按下 Animation 視窗的 **Play** 鍵預覽。
 - 如果太快：調整 **Samples** (預設 60，通常改 12~24 比較剛好) 。

錄製動畫：Run (跑步)

1. 在 Animation 視窗左上角的下拉選單 (現在顯示 Player_Idle)。
2. 選擇 **Create New Clip...**。
3. 命名為 **Player_Run**。
4. 同樣步驟，把跑步的連續圖拖進去。
5. 調整 Samples 速度，讓它跑起來自然一點。

錄製動畫：Jump (跳躍)

1. Create New Clip -> **Player_Jump** 。
2. 把跳起來的圖拖進去。
 - 通常跳躍只有一張圖，或是「起跳 -> 滯空 -> 落地」三張。
 - 如果是單張圖，就只拖那一張即可。

觀察 Animator 元件

當你建立第一個 Animation 時，Unity 自動幫主角加了一個 **Animator** 元件。
並且建立了一個 **Animator Controller** 檔案。

- **Animation Clip**：動作片段 (MP4)。
- **Animator Controller**：大腦，決定現在要播哪一支片 (播放器)。

設定狀態機 (Animator Window)

1. 開啟 Animator 視窗 (Window -> Animation -> Animator)。
2. 選取主角，你會看到三個方塊：
 - Entry (綠色)：入口。
 - Player_Idle (橘色)：預設狀態。
 - Player_Run (灰色)。
 - Player_Jump (灰色)。

(如果你的 Run 是橘色的，在 Idle 上按右鍵 -> Set as Layer Default State)

建立參數 (Parameters)

動畫切換需要條件。我們要設定「參數」讓程式控制。

在 Animator 視窗左側的 **Parameters** 分頁，點  新增：

1. **Speed** (Float)：代表移動速度 (0=不動, >0=跑)。
2. **IsGround** (Bool)：代表是否在地板上 (True=跑/站, False=跳)。

建立過渡 (Transitions)：站 <-> 跑

我們希望：速度 > 0.1 就跑，速度 < 0.1 就停。

1. 在 Idle 按右鍵 -> Make Transition -> 連到 Run。

2. 點選這條白線，看 Inspector：

- Has Exit Time：取消勾選 (不然會等動作播完才切，會有延遲)。
- Transition Duration：設為 0 (2D 遊戲通常瞬間切換)。
- Conditions：新增 **Speed** -> **Greater** -> **0.1**。

3. 同樣步驟做回來 (Run -> Idle)：

- Conditions：**Speed** -> **Less** -> **0.1**。

建立過渡：任意狀態 -> 跳

跳躍應該是隨時都可以發生的 (Any State)。

1. 在 **Any State** (淡藍色方塊) 按右鍵 -> 連到 **Jump**。

2. 設定條件：

- Has Exit Time: 關閉。
- Conditions: **IsGround** -> **False**。

3. 跳完何時回來？

- 從 **Jump** 連回 **Idle**。
- Conditions: **IsGround** -> **True**。

程式控制 (Coding)

回到 `PlayerController.cs`，我們要告訴 Animator 現在的情況。

1. 宣告變數：

```
public Animator anim;
```

2. 在 Start 抓取：

```
anim = GetComponent<Animator>();
```

傳送參數

在 `Update()` 裡面，把物理數值傳給 Animator：

```
void Update()
{
    // ... 原本的輸入邏輯 ...

    // 1. 設定速度（取絕對值，因為往左往右都要算跑）
    anim.SetFloat("Speed", Mathf.Abs(rb.velocity.x));

    // 2. 設定是否在地板
    anim.SetBool("IsGround", isGrounded);
}
```

修正：Run 動畫播放速度

如果我跑得慢，動畫應該播慢一點？

1. 選取 Animator 視窗裡的 **Run** 狀態。
2. 勾選 **Parameter** (在 **Speed** 欄位旁)。
3. 選擇 **Speed** 參數。
4. 現在動畫播放速度會跟著你的移動速度連動了！

常見問題：卡在 Run 狀態？

Q: 我放開按鍵了，角色停住了，但動畫還在原地踏步？

A:

1. 檢查 `rb.velocity.x` 是否真的歸零？
 - 有時候物理會有殘留微小速度 (0.0001)。
 - 可以把過渡條件 `Speed > 0.1` 改大一點。
2. 檢查 Transition 的 **Has Exit Time** 是否真的關掉了？

常見問題：跳躍切換慢？

Q: 跳起來過了一下子才變跳躍姿勢？

A:

這通常是因為 **Any State -> Jump** 的 Transition Duration 不是 0。

2D 遊戲講求反應快，請務必把 Duration 設為 0。

混合樹 (Blend Tree)

進階補充

如果你的遊戲有「走 -> 小跑 -> 快跑」，不想連一堆線...

1. 右鍵 Create State -> **From New Blend Tree**。
2. 雙擊進入。
3. 設定 Threshold (閾值)，根據 Speed 自動混合不同的動畫片段。
(本課程簡單版只需 Idle/Run 切換即可)

下樓梯問題

在膠囊體下樓梯時，有時候會有短暫的「騰空」。

導致 `isGrounded` 瞬間變 `false` -> 觸發 Jump 動畫 -> 導致角色在那邊抽蓄。

解法：

在 Jump 的 Transition 增加 **Transition Duration** (例如 0.1秒)。

如果是極短暫的離地，還來不及切換到 Jump 就又著地了，可以掩蓋這個問題。

總結

動畫是遊戲的靈魂。

1. Animation Clip：錄製動作。
2. Animator Controller：規劃狀態流程。
3. Parameters：溝通橋樑。
4. Code：`anim.SetFloat` , `anim.SetBool` 。

現在你的主角不僅能動，還動得很有生命力！

下週預告

角色跟場景都完美了，但...

「我又不知道我有多少錢？」

「我也看不到血量？」

下週我們進入 UI 介面設計：

- Canvas (畫布)。
- TextMeshPro (高品質文字)。
- 製作血條與計分板。

Q & A

- 可以做攻擊動畫嗎？
 - 可以，新增一個 Trigger 參數 **Attack** ◦
 - 在 Any State 連到 Attack (Condition: Attack) ◦
 - Attack 連回 Idle (Has Exit Time: True，讓它播完) ◦
- 動畫圖檔切割有問題？
 - 檢查 Sprite Mode: Multiple 和 Sprite Editor ◦

(助教巡堂協助)